

AEM (Solace) Messaging Connection

File: `srv/messaging/aemmessaging.ts` | CollabSphere — SAP CAP Backend

Purpose	Establishes a session to the AEM (Solace PubSub+ / Advanced Event Mesh) broker and binds a Guaranteed-Message consumer to the queue <code>Q. EMPLOYEE.COLLABSPHERE</code> to receive employee-related messages (e.g. from an upstream MDM system).
Library	<code>solclientjs</code> — the Solace JavaScript Messaging API
Trigger	The connection is opened as a module-level side effect — simply importing this file starts the connection attempt.

Contents

- Section 1 — Full Source Code
- Section 2 — Line-by-Line Explanation
 - 2.1 Imports & Environment Variables
 - 2.2 Environment Variable Validation
 - 2.3 SDK Logger Setup
 - 2.4 Solace Factory Initialization
 - 2.5 Session Creation
 - 2.6 Session Event: Successful Connection (UP_NOTICE)
 - 2.7 Queue Descriptor & Message Consumer
 - 2.8 Message Consumer Events (UP / DOWN_ERROR / MESSAGE)
 - 2.9 Session-Level Failure & Disconnect Events
 - 2.10 Triggering the Connection
 - 2.11 Error Handling & Module-Level Invocation

Section 1 — Full Source Code

```
import solace from "solclientjs";

function aemCreateSession() {
  const host = process.env.AEM_HOST;
  const vpn = process.env.AEM_VPN;
  const username = process.env.AEM_USERNAME;
  const password = process.env.AEM_PASSWORD;
  const queue = process.env.AEM_QUEUE;

  try {
    // Validate environment variables
    if (!host || !vpn || !username || !password || !queue) {
      throw new Error(
```

```

    "Missing required environment variables for AEM connection.",
  );
}

// Solace-compatible logger
const logger = new solace.LogImpl(
  console.trace.bind(console),
  console.debug.bind(console),
  console.info.bind(console),
  console.warn.bind(console),
  console.error.bind(console),
  console.error.bind(console),
);

// Initialize Solace JavaScript API
solace.SolclientFactory.init({
  profile: solace.SolclientFactoryProfiles.version10,
  logLevel: solace.LogLevel.INFO,
  logger: logger,
});

console.log("Solace SDK initialized.");

// Create Solace session
const session = solace.SolclientFactory.createSession({
  url: host,
  vpnName: vpn,
  userName: username,
  password: password,
});

console.log("Solace session created.");

// Successful connection
session.on(solace.SessionEventCode.UP_NOTICE, () => {
  console.log("Successfully connected to AEM!");

  const queueDescriptor = new solace.QueueDescriptor({
    name: queue,
    type: solace.QueueType.QUEUE,
  });

  console.log(`Queue descriptor created: ${queue}`);

  // Create message consumer bound to the queue
  const messageConsumer = session.createMessageConsumer({
    queueDescriptor: queueDescriptor,
    acknowledgeMode: solace.MessageConsumerAcknowledgeMode.CLIENT,
  });

  console.log("Message consumer created.");

  // Queue connected successfully
  messageConsumer.on(solace.MessageConsumerEventName.UP, () => {
    console.log(`Successfully connected to queue: ${queue}`);
  });
}

```

```

    });

    // Queue connection failed
    messageConsumer.on(
        solace.MessageConsumerEventName.DOWN_ERROR,
        (error) => {
            console.error("Queue connection failed:", error);
        },
    );

    // Message received
    messageConsumer.on(solace.MessageConsumerEventName.MESSAGE, (message) => {
        const payload = message.getBinaryAttachment()?.toString();
        console.log("Message received:", payload);
        message.acknowledge();
    });

    // Connect consumer to queue
    console.log(`Connecting to queue: ${queue}`);

    messageConsumer.connect();
});

// Connection failure
session.on(solace.SessionEventCode.CONNECT_FAILED_ERROR, (error) => {
    console.error("Failed to connect to AEM:", error);
});

// Disconnection
session.on(solace.SessionEventCode.DISCONNECTED, () => {
    console.log("Disconnected from AEM.");
});

// Connect to AEM
console.log("Connecting to AEM...");
session.connect();
} catch (error: any) {
    console.error("Error connecting to AEM messaging service:", error.message);
}
}

export { aemCreateSession };

aemCreateSession();

```

Section 2 — Line-by-Line Explanation

2.1 — Imports & Environment Variables

```
import solace from "solclientjs";
```

Why: Loads the Solace Messaging API for Node.js. This single import gives access to every class/enum used in the file — `SolclientFactory`, `Session`, `QueueDescriptor`, `MessageConsumer`, and the related event-name enums.

Value given: The `solace` namespace object, used throughout the file as the entry point into the SDK.

```
function aemCreateSession() {  
  const host = process.env.AEM_HOST;  
  const vpn = process.env.AEM_VPN;  
  const username = process.env.AEM_USERNAME;  
  const password = process.env.AEM_PASSWORD;  
  const queue = process.env.AEM_QUEUE;
```

Why: Keeps broker connection details out of source code. Credentials and environment-specific endpoints (dev vs. prod broker) must never be hard-coded — they come from `.env` / the deployed environment instead.

Value given:

- `host` — the broker's WebSocket URL, e.g. `wss://mr-connection-...messaging.solace.cloud:443`
- `vpn` — the Solace "Message VPN", a logical partition/tenant on the broker (not a network VPN)
- `username` / `password` — the client credentials the broker authenticates against
- `queue` — the name of the Guaranteed-Message queue this service binds to (`Q.EMPLOYEE.COLLABSPHERE`)

2.2 — Environment Variable Validation

```
try {  
  // Validate environment variables  
  if (!host || !vpn || !username || !password || !queue) {  
    throw new Error(  
      "Missing required environment variables for AEM connection.",  
    );  
  }  
}
```

Why: Fails fast with a clear message if any required variable is missing, instead of letting the SDK fail later with a confusing low-level error (e.g. attempting to connect to `undefined`).

Value given: A guarantee that everything below this point runs only with a complete, valid configuration.

2.3 — SDK Logger Setup

```
// Solace-compatible logger
const logger = new solace.LogImpl(
  console.trace.bind(console),
  console.debug.bind(console),
  console.info.bind(console),
  console.warn.bind(console),
  console.error.bind(console),
  console.error.bind(console),
);
```

Why: The Solace SDK needs a logger to report its own internal activity (connection state changes, transport events, errors). It does not accept a plain object — it requires an actual instance of `solace.LogImpl`, built with six positional callback arguments in this exact order: `trace`, `debug`, `info`, `warn`, `error`, `fatal`. (An earlier version of this file passed a plain object missing `trace`, which the SDK rejected with a “Parameter logger.fatal/trace failed validation” error at init time.)

Value given: Each parameter is a console method bound to the `console` object (so `this` resolves correctly when the SDK calls them standalone). There is no `console.fatal`, so `console.error` is reused for the `fatal` slot. The result, `logger`, is handed to `SolclientFactory.init()` below.

2.4 — Solace Factory Initialization

```
// Initialize Solace JavaScript API
solace.SolclientFactory.init({
  profile: solace.SolclientFactoryProfiles.version10,
  logLevel: solace.LogLevel.INFO,
  logger: logger,
});

console.log("Solace SDK initialized.");
```

Why: One-time, global SDK bootstrap that must happen before any session is created.

Value given:

- `profile: version10` — selects the current recommended API behavior profile (as opposed to `version7`, kept only for legacy compatibility)
- `logLevel1: INFO` — filters the SDK's own internal logs so only `info` -and-above messages reach the logger (suppresses noisy `trace` / `debug` chatter)
- `logger` — the `LogImpl` instance built in 2.3

The `console.log` line is a plain application-level breadcrumb, unrelated to the SDK's own logger — it just confirms this step completed.

2.5 — Session Creation

```
// Create Solace session
const session = solace.SolclientFactory.createSession({
  url: host,
  vpnName: vpn,
  userName: username,
  password: password,
});

console.log("Solace session created.");
```

Why: Builds a `Session` object configured with where to connect and how to authenticate. This step is purely local — it does **not** open a network connection yet.

Value given: `session` — the object every subsequent call (`.on(...)`, `.connect()`, `.createMessageConsumer(...)`) is made against.

2.6 — Session Event: Successful Connection (UP_NOTICE)

```
// Successful connection
session.on(solace.SessionEventCode.UP_NOTICE, () => {
  console.log("Successfully connected to AEM!");
  ...
});
```

Why: Registers a listener for the event the SDK fires once the session has actually connected *and* authenticated against the broker. Everything queue-related is deliberately nested inside this callback, because binding a consumer only makes sense once the underlying session is live.

Value given: A guaranteed ordering — the queue consumer is only created after the broker has confirmed the session is up, never before.

2.7 — Queue Descriptor & Message Consumer

```
const queueDescriptor = new solace.QueueDescriptor({
  name: queue,
  type: solace.QueueType.QUEUE,
});

console.log(`Queue descriptor created: ${queue}`);

// Create message consumer bound to the queue
const messageConsumer = session.createMessageConsumer({
  queueDescriptor: queueDescriptor,
  acknowledgeMode: solace.MessageConsumerAcknowledgeMode.CLIENT,
});

console.log("Message consumer created.");
```

Why: To receive Guaranteed Messages from a queue, the SDK requires a `QueueDescriptor` identifying the queue by name and type, then a `MessageConsumer` bound to that descriptor via `session.createMessageConsumer(...)`. (Earlier drafts of this code incorrectly used `session.createFlow(...)` and `createQueueDestination(...)` — neither exists on the SDK; `createQueueDestination` builds an object for *publishing to* a queue, not for *consuming from* one.)

Value given:

- `name: queue` — the exact queue name on the broker, `Q. EMPLOYEE.COLLABSPHERE` (must match character-for-character, including the space, or the broker returns `503 UNKNOWN_QUEUE_NAME`)
- `type: solace.QueueType.QUEUE` — distinguishes an actual queue endpoint from a topic endpoint
- `acknowledgeMode: CLIENT` — messages are only removed from the queue when the application explicitly calls `message.acknowledge()` (as opposed to `AUTO`, where the SDK acks automatically once all listeners return without throwing)

2.8 — Message Consumer Events (UP / DOWN_ERROR / MESSAGE)

```
// Queue connected successfully
messageConsumer.on(solace.MessageConsumerEventName.UP, () => {
  console.log(`Successfully connected to queue: ${queue}`);
});

// Queue connection failed
messageConsumer.on(
  solace.MessageConsumerEventName.DOWN_ERROR,
  (error) => {
    console.error("Queue connection failed:", error);
  },
);

// Message received
messageConsumer.on(solace.MessageConsumerEventName.MESSAGE, (message) => {
  const payload = message.getBinaryAttachment()?.toString();
  console.log("Message received:", payload);
  message.acknowledge();
});
```

Why: A `MessageConsumer` is an event emitter; nothing happens unless listeners are attached before connecting.

Value given:

- `UP` — fires once the consumer has successfully bound to the queue on the broker.
- `DOWN_ERROR` — fires if the bind fails (e.g. the queue doesn't exist, or the client lacks permission). Logging `error` here surfaces the broker's reason (e.g. `503 UNKNOWN_QUEUE_NAME`).
- `MESSAGE` — fires for every message delivered from the queue. `message.getBinaryAttachment()` returns the raw payload as a Node `Buffer`; `.toString()` decodes it to the original UTF-8/JSON text that was published. `message.acknowledge()` tells the broker the message was successfully handled and can be removed from the queue — without this call (given `acknowledgeMode: CLIENT`), the message stays spooled on the broker and gets redelivered.

```
// Connect consumer to queue
console.log(`Connecting to queue: ${queue}`);

messageConsumer.connect();
});
```

Why: Listeners are attached first, *then* the actual bind request is sent — this ordering avoids a race where the `UP` or `MESSAGE` event could fire before anything is listening for it.

Value given: Kicks off the (asynchronous) queue bind. The outcome arrives later via the `UP` / `DOWN_ERROR` listeners registered just above.

2.9 — Session-Level Failure & Disconnect Events

```
// Connection failure
session.on(solace.SessionEventCode.CONNECT_FAILED_ERROR, (error) => {
  console.error("Failed to connect to AEM:", error);
});

// Disconnection
session.on(solace.SessionEventCode.DISCONNECTED, () => {
  console.log("Disconnected from AEM.");
});
```

Why: Mirrors 2.6/2.8 but at the session level rather than the queue level — these cover failures/events for the underlying broker connection itself (e.g. bad credentials, which surfaces here as a `401 Unauthorized` / login-failure error), separate from anything queue-specific.

Value given: Visibility into session-level connectivity problems (auth failures, network drops) independent of whether a queue consumer was ever successfully created.

2.10 — Triggering the Connection

```
// Connect to AEM
console.log("Connecting to AEM...");
session.connect();
```

Why: Everything above this line only *configures* the session and its listeners. This line performs the actual network connect + login attempt.

Value given: `session.connect()` is non-blocking — it returns immediately, and the real outcome (success or failure) is reported later through the `UP_NOTICE` / `CONNECT_FAILED_ERROR` event handlers, not through a return value here.

2.11 — Error Handling & Module-Level Invocation

```
} catch (error: any) {
  console.error("Error connecting to AEM messaging service:", error.message);
}
}

export { aemCreateSession };

aemCreateSession();
```

Why: The `catch` block only catches *synchronous* setup errors thrown inside the `try` (e.g. the missing-env-var throw in 2.2, or a malformed config object rejected immediately by `init / createSession`). It does **not** catch asynchronous failures like a login rejection or queue bind failure — those are only visible through the event listeners registered earlier (2.6, 2.8, 2.9).

Value given:

- `export { aemCreateSession }` — makes the function importable elsewhere, in case another module ever needs to trigger it explicitly.
- `aemCreateSession();` — immediately invokes the function as a module-level side effect. This means simply `import`-ing this file anywhere in the codebase starts a live connection attempt to AEM — there is no separate explicit call site elsewhere that “starts” messaging.